

Assessment: JavaFX UI Design Challenge – “Serenity” Wellness Dashboard

Objective

Design and implement **only the user interface** for the **Home Screen** and a **Habit Detail Overlay** of a fictional wellness app called *Serenity*. No backend, no real data—hardcoded sample data is allowed. The goal is to demonstrate mastery of JavaFX layout, custom drawing (Canvas), animations (Timeline, Transition), effects (DropShadow, BoxBlur, PerspectiveTransform), and interaction design.

Your implementation must follow the exact specifications below. The complexity is intentionally high; superficial AI-generated code will not satisfy all the detailed visual and interactive requirements.

Scenario

Serenity is a daily wellness companion. The Home Screen gives an at-a-glance view of emotional state, daily goal progress, a motivational carousel, and an interactive grid of wellness habits. Tapping a habit opens a blurred overlay that slides up from the bottom with detailed statistics and a quick-log action. The entire experience is fluid, with custom-painted graphics, perspective effects, and coordinated animations.

Design Specifications

Colour Palette

Role	Hex Code (use JavaFX <code>Color.web()</code>)
Primary Background	<code>#F9F7F7</code> (soft warm white)

Role	Hex Code (use JavaFX <code>Color.web()</code>)
Surface Cards	<code>#FFFFFF</code> with drop shadow (black 8% opacity)
Primary Accent	<code>#6C63FF</code> (vibrant purple)
Secondary Accent	<code>#FF6584</code> (coral pink)
Success	<code>#2EC4B6</code>
Warning	<code>#FFBF46</code>
Text Primary	<code>#2D2D2D</code>
Text Secondary	<code>#7A7A7A</code>
Glassmorphism (overlay)	<code>rgba(255,255,255,0.85)</code> + <code>BoxBlur</code> radius 12

Typography

- **App title / large numbers:** "Playfair Display" (load via `Font.loadFont()`), weight bold, letter spacing -0.5
- **Headings (card titles, section headers):** "System" font family with weight `FontWeight.BOLD` (simulate SF Pro Rounded)
- **Body, labels, captions:** "System" regular, size 12–14
- **Mood ring text:** "Dancing Script" (load from Google Fonts), bold, italic

Spacing & Dimensions

- Root `BorderPane` padding: 24 px (left, right), 16 px (top, bottom)
- Card corner radius: 20 px (background with `Background` + `BackgroundFill` + `CornerRadii`)
- Circular progress: diameter 160 px, stroke width 14 px
- Habit grid: `TilePane` `columns=2`, `tileAlignment=Pos.CENTER`, each tile width constrained, aspect ratio 0.85
- Habit card perspective effect: rotation around X and Y axes (max $\pm 12^\circ$ for X, $\pm 8^\circ$ for Y) based on scroll position, implemented with `PerspectiveTransform` effect
- Bottom overlay: a `StackPane` that slides up from bottom, default visible height 180 px, expands to 65% of window height on drag. It uses `BoxBlur` on a background snapshot.

- Overlay avatar: 70 px circle, positioned half above the overlay's top edge (`translateY = -35`)
 - Bottom navigation: custom `HBox` height 72 px with a top-notch cutout (radius 36 px) for a floating action button
-

Screen Layout & Required Elements (JavaFX Hierarchy)

Create a single `Stage` containing a `Scene`. The root is a `BorderPane` with a semi-transparent top area, a center `ScrollPane` (with pull-to-refresh custom header), and a bottom custom navigation bar. The overlay (habit detail) is a separate `StackPane` managed via a `Dialog` or by adding/removing from the root's children with animated translation.

1. Top Bar (custom `HBox` inside top region)

- Transparent background.
- Left: circular `ImageView` (40 px, placeholder `avatar.png` clipped to circle).
- Right: a `StackPane` containing a `Label` (bell icon `\uD83D\uDD14`) and a red circle badge (6 px radius) when `unread = true`.
- Center: current date in size 12, colour Secondary Text, format "Wednesday, 3 Jun".

2. Hero Section (inside `ScrollPane` content, a `VBox`)

a) Mood Ring (`Canvas` 120×120)

- Draw a conic gradient stroke (width 12) using `GraphicsContext`'s `setStroke` with a custom `LinearGradient`? (JavaFX lacks conic gradient natively; you must implement it by segmenting the arc into small pieces or using a pre-computed image).
- A glowing shadow: draw a second circle with larger radius, same colour with opacity 40%, and apply `BoxBlur(18)`.
- Inside, display the current mood word ("Calm", "Focused" etc.) in `Dancing Script` 18, Primary Accent.
- Handle mouse drag on the ring to convert angle to a 0–1 value, updating a local `DoubleProperty`.
- The glow pulsates opacity continuously between 0.3 and 0.6 (use `Timeline`).

b) Daily Goal Progress (`Canvas` 160×160)

- Draw a background track (grey, rounded caps).

- Draw a progress arc using a sweep gradient (Primary→Secondary) with rounded caps.
- Apply a glow effect to the arc (draw a blurred copy behind).
- Center text: "78%" in Playfair Display 32, Primary Text; below "of daily goal" in System 11, Secondary Text.
- Animate the arc from angle 0 to $0.78 * 360$ using `Timeline` (1.2 s, `Interpolator.EASE_OUT`).

3. Motivational Quote Carousel (custom `ScrollPane` or `PageView`-like behaviour)

- Create three quote cards (width: 85% of window width, height 100 px), styled with rounded corners and a subtle gradient overlay.
- Implement a horizontal scroll that snaps to cards. While scrolling, the centre card scales to 1.0, others scale to 0.92 and shift down by 6 px (parallax stack effect). Use a `ScrollPane` with custom scroll event listener and manual snapping.
- Auto-advance every 4 seconds using `Timeline`, looping.

4. Habit Categories (horizontal `HBox` inside a `ScrollPane`)

- 7 circular category chips (diameter 60 px) with icon and gradient background.
- Below each chip, a label (System 10, Secondary Text).
- The selected chip (default index 2) has a scale animation (1.15) and white border 2 px. Only one selectable.
- Selection changes cause the habit grid below to cross-fade to a new set of habit cards (use `FadeTransition` on the grid's parent `Group` or `StackPane`).

5. Habit Grid with Perspective Effect

- A `TilePane` (2 columns) inside the `ScrollPane` content.
- Four hardcoded habits: "Meditation", "Hydration", "Reading", "Stretching".
- Each habit card is a `Region` with rounded corners, drop shadow, and inner gradient overlay. Contains habit icon (32 px), title (System 18), and streak fire emoji + number (bottom right).
- **Perspective tilt:** As the card's position in the viewport changes (track via `boundsInParent` and `ScrollPane.vvalue`), rotate the card around X and Y axes using a `PerspectiveTransform` effect. Rotation X: 0° at top/bottom to -12° at centre; Y: -8° (left) to +8° (right). Use an `AnimationTimer` or `ChangeListener` on scroll position, apply smooth interpolation (lerp factor 0.15).
- On mouse click, trigger a custom **hero transition**: the card's icon moves from its location to the overlay's avatar position using `TranslateTransition` and `ScaleTransition`; simultaneously the overlay slides up.

6. Bottom Navigation Bar (custom Region or HBox in bottom slot)

- Height 72 px, with a **top-notch cutout** at the centre (radius 36 px). Draw this shape using Canvas or an SVGPath for the background.
- A FloatingActionButton (circle 56 px, Primary Accent, "+") sits inside the notch, with drop shadow.
- Three navigation icons on each side (home, insights, profile), spaced evenly, with the active one tinted Primary Accent and scaled 1.2.
- Frosted glass effect: clip a snapshot of the background area, apply BoxBlur(8), and use it as the bar's background (see Robot or Node.snapshot).

7. Habit Detail Overlay (a StackPane sliding up from bottom)

- Not a separate window; add it to the root StackPane (overlying the main BorderPane).
 - Default state: translated so that only top 180 px is visible. Drag up to expand up to 65% of stage height. Implement drag via mouse events on a "handle" area.
 - The overlay has a frosted glass background: take a snapshot of the entire scene behind, blur it, and set as background image.
 - The hero avatar (habit icon in a 70 px circle) is positioned centred horizontally, overlapping the overlay's top edge by 35 px. Animate it from the habit card to this position when opening.
 - Content (scrollable):
 - Habit name in Playfair Display 24.
 - "Current Streak" with flame icon and large number (36 px, Success).
 - A **custom painted line chart** (Canvas height 160 px) showing 7 days of hardcoded data. Line: gradient stroke (Primary→Secondary) 3 px width, with circular data points (radius 4). Grid lines dashed. Animate the line path from left to right on overlay appear (1 s, Interpolator).
 - "Quick Log" button (pill shaped, full width, gradient). On click, a **confetti burst** animation plays: draw 20 small coloured circles that fly outward and fade out using Timeline (800 ms). Purely visual.
 - Overlay closing animation: reverse the slide-down and hero transition.
-

Animations & Micro-interactions (Mandatory)

1. **Mood ring glow** pulsates (opacity oscillation).

2. **Circular progress** arc fills with ease-out.
 3. **Quote carousel** auto-advances with scale/parallax snap.
 4. **Category chip** selection scales with a spring-like `Timeline` (use `Interpolator.SPLINE(0.25, 0.1, 0.25, 1)`).
 5. **Habit card perspective** updates smoothly at 60 fps (using `AnimationTimer` and `lerp`).
 6. **Hero animation**: icon moves and scales from card to overlay avatar (sync with overlay slide).
 7. **Line chart** draws path progressively.
 8. **Confetti burst** on button click.
 9. **Pull-to-refresh**: When the `ScrollPane` is pulled down more than 80 px at the top, show a custom **morphing loader** (a circle that splits into three smaller circles, rotates, and reforms) drawn on a `Canvas`. Use mouse drag events to track overscroll, and upon release, run the morphing animation for 2 seconds then reset.
-

Custom JavaFX Components You Must Build (Reusable)

- `MoodRing` (Region using `Canvas`)
 - `GlowCircularProgress`
 - `HabitCard3D` (extends `Region`, accepts title, icon, streak, hero tag)
 - `MorphingRefreshIndicator` (custom `Node` placed above the `ScrollPane`)
 - `CurvedBottomNavBar` (with notch background drawing)
 - `AnimatedLineChart` (extends `Canvas`)
 - `GlassmorphicBackground` (utility to apply blur effect to a background snapshot)
-

Assets (Placeholders)

Include these in a `resources/` folder:

- `avatar.png`
- All icons can be emoji characters or built-in JavaFX SVG paths, but the final appearance must match the described styling.
- For fonts, download Playfair Display and Dancing Script from Google Fonts and load them via `Font.loadFont`.

Constraints

- **Pure UI** – buttons trigger animations/overlays but do not store/process data. All data is hardcoded.
 - **JavaFX version:** 17+ (with JavaFX modules properly added).
 - **No external UI libraries** – only standard JavaFX (`javafx.controls`, `javafx.graphics`, `javafx.animation`, etc.).
 - **No scene builder / FXML** – all UI must be built programmatically to show layout mastery.
 - AI code generation tools may be used, but the assignment's highly specific, interconnected custom painting, perspective transforms, and choreographed animations make it extremely difficult to obtain a fully correct solution from a simple prompt.
-

Submission

1. Complete JavaFX project (Maven/Gradle or plain with jar) including all source files, resources, and a runnable JAR.
 2. A short screen recording (GIF or MP4) demonstrating:
 - Scrolling the home screen, perspective tilt visible.
 - Mood ring drag interaction and glow pulsation.
 - Category chip selection and grid cross-fade.
 - Hero animation to detail overlay.
 - Pull-to-refresh morphing loader.
 - Confetti burst.
-

Evaluation Rubric (Total: 100 points)

Category	Points	Details
Layout Fidelity	25	Exact match to spacing, sizing, and composition.
Custom Painting & Effects	25	Quality of MoodRing, GlowCircularProgress, AnimatedLineChart, MorphingRefresh Indicator, glassmorphism, conic gradient workaround, glow, perspective transform.
Animations & Interactions	25	Smoothness, correct interpolators, hero transition, overlay drag, perspective tilt smoothing, confetti.
Code Structure & Reusability	15	Clean separation, custom component reusability, efficient listeners.
Overall Polish & Creativity	10	Attention to typography, shadows, glass effect, creative fidelity within constraints.